



4



12



1800 sp

عملي

محتوى مجاني غير مخصص للبيع التجاري

كلية الهندسة المعلوماتية

السنة الرابعة

## CRONTAB AND IP ADDRESSES

عملي مشترك



نظم التشغيل

**RB** Informatics; 11/11/2025

- كثيراً ما نواجه في بيئة العمل الحاجة إلى تنفيذ مهام متكررة خلال فترات زمنية محددة، مثل إجراء نسخ احتياطي للبيانات كل يوم سبت في ساعة معينة. ومع تكرار هذه المهام تصبح عملية تنفيذها يدوياً مرهقة وغير عملية، خصوصاً عند ازدياد عددها.
- هنا تبرز أهمية جدولة المهام في لينكس، والتي تتيح لنا تحديد المهمة المراد تنفيذها والأوقات الزمنية المحددة لعملها، ليقوم النظام بتنفيذها تلقائياً دون أي تدخل يدوي، ويتم ذلك من خلال أداة Cron، التي تمثل محور محاضرتنا لهذا اليوم.

## CronTab

ما هو ال cron ؟

- Cron هو مجدول مهام (Job Scheduler) في لينكس يُستخدم لجدولة تشغيل الأوامر والسكريبتات بشكل دوري وفق وقت ثابت أو فاصل زمني محدد.
- يُسمى كل أمر أو سكريبت يتم تشغيله بواسطة Cron بال "Cron Job".
- مهام الـ Cron هي أوامر أو سكريبتات Shell يتم تسجيلها داخل ملفات crontab.
- يقوم النظام بتحميل هذه الملفات في الذاكرة، ويقوم خادم الكرون (Cron daemon) بمراقبة هذه الملفات وتشغيل الأوامر في الأوقات المحددة لها، حيث يستيقظ الـ Cron كل دقيقة ليفحص جميع ملفات الـ crontab المخزنة، ويتحقق مما إذا كان هناك أي أمر يجب تنفيذه في تلك الدقيقة تحديداً.



**المعرفة تطهى بالجهد والصبر**

## ملفات Crontab

- Crontab، المعروف أيضاً باسم جدول Cron ، هو قائمة بإعدادات البيئة وأوامر Cron المخزنة في ملف ويستخدمها خادم Cron لتنفيذ المهام على أساس جدول. هناك أنواع مختلفة من ملفات crontab متاحة.
- لكل مستخدم ملف crontab خاص به يتم تخزينه في منطقة spool ويسمى باسم حساب المستخدم، يتم تنفيذ كل crontab باسم المستخدم الذي يملكه.
- يمكننا التحقق من ملفات crontab المملوكة للمستخدم عن طريق عرض جميع الملفات في دليل spool باستخدام الأمر:

```
sudo ls /var/spool/cron/crontabs/
```

```
rbcs@tasneem-VMware-Virtual-Platform:~$ sudo ls /var/spool/cron/crontabs
[sudo] password for rbcs:
rbcs  usertest
```

نلاحظ من الصورة أنه لدينا مستخدمين لهما ملف crontab

## إدارة ملفات Crontab

- لفتح و تعديل الملف نستخدم الأمر: `crontab -e` حيث `e` تعني editing إذا لم يكن ملف crontab موجوداً مسبقاً للمستخدم، يقوم هذا الأمر بإنشائه تلقائياً، ويطلب عندها تحديد نوع المحرر النصي الذي سيتم استخدامه في كل مرة يتم فيها تنفيذ الأمر. في محاضراتنا سنعتمد على المحرر vim.basic

```
rbcs2@tasneem-VMware-Virtual-Platform:~$ crontab -e
no crontab for rbcs2 - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 2
```

- لعرض محتوى ملف ال crontab بدون تعديله : `crontab -l`
- لحذف ملف ال crontab : `crontab -r -i`

جميع الأوامر السابقة تتعامل مع ملف crontab الخاص بالمستخدم الحالي ، للتعامل مع ملف crontab الخاص بالمستخدم الجذر (root) ، نستخدم نفس الأوامر مع اضافة sudo في بدايتها .

فائدة اريسيزية : Shell Scripting

## ■ ما هو Shell Script ؟

1. هو ملف يحتوي على مجموعة من الأوامر المكتوبة بطريقة منظمة لتنفيذ مهام تلقائياً اي لا نحتاج لكتابة الأوامر ذاتها في كل مرة .
2. امتداد الملفات غالباً .sh

## ■ أهمية ال Shell Script :

1. أتمتة المهام المتكررة (مثل النسخ الاحتياطي أو تثبيت البرامج).
2. تسريع العمليات على النظام مقارنة بتنفيذ الأوامر يدوياً.

مثال بسيط :

```

$ !/bin/bash Untitled-1
1  !/bin/bash
2
3  echo "hello world" >> test/test.txt
4  |

```

هذا ملف بسيط يحوي على أمر لكتابة hello world في ملف اخر يدعى test.txt .

سنتعرف أكثر على مفهوم ال shell scripting في المحاضرة القادمة ولكن القينا لمحة لأنه سيرد معنا في هذه المحاضرة .

## مهام Cron

## ■ فهم صياغة مهام ال cron :

- تكتب كل مهمة Cron باستخدام تعبير يتكون من جزئين : جدول الوقت والأمر الذي سيتم تنفيذه, ويمكن أن يكون الأمر أي أمر يتم تنفيذه عادة في ال terminal .
- تتكون صيغة مهمة Cron من 6 عناصر تفصل بينها الفراغات. العناصر الخمسة الأولى تصف وقت التنفيذ, بينما العنصر الأخير هو الأمر نفسه أو المسار الكامل لملف shell script الذي سيتم تنفيذه بواسطة ال shell الافتراضية.

[الأمر] [يوم من الأسبوع] [الشهر] [يوم من الشهر] [الساعات] [الدقائق]



| الحقل          | القيم المسموحة                                                                              |
|----------------|---------------------------------------------------------------------------------------------|
| الدقائق        | 0 - 59                                                                                      |
| الساعات        | 0 - 23 (تتبع نظام ال 24)                                                                    |
| يوم من الشهر   | 1 - 31                                                                                      |
| الشهر          | 1 - 12<br>حيث 1 يقابل الشهر الأول وهكذا, ويمكن استخدام اسم الشهر بالأحرف الكبيرة: JAN - DEC |
| يوم من الأسبوع | 0 - 6<br>حيث 0 يقابل يوم الأحد وهكذا, ويمكن استخدام اسم اليوم بالأحرف الكبيرة: SUN - SAT    |

يتم تنفيذ الأوامر بواسطة Cron عندما تتطابق قيم الدقيقة والساعة والشهر مع الوقت الحالي، وعندما يتطابق أحد حقول اليوم على الأقل، سواء كان يوم من الشهر أو يوم من الأسبوع، مع الوقت الحالي.

#### بعض الرموز الخاصة التي يمكن استخدامها لتحديد وقت التنفيذ بشكل أدق:

| الرمز | الشرح                                                                                                                                                                                                                                          |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| *     | في تعابير cron تمثل النجمة متغيراً يدلّ على الكلّ. لذلك، فإنّ مهمة مجدولة باستخدام ( * * * * * ) ستعمل كل دقيقة من كل ساعة من كل يوم من كل شهر.                                                                                                |
| ,     | تستخدم الفاصلة لتجميع قيم الجدولة لتكوين قائمة. فإذا رغبت في تشغيل مهمة في بداية كل ساعة ووسطها دون كتابة مهمتين منفصلتين مثل ( ..... * * * * 0 و ..... * * * * 30 ) يمكن تحقيق النتيجة نفسها بصيغة واحدة (0,30 * * * * )                      |
| -     | يشير هذا الرمز إلى مدى من القيم داخل حقل الجدولة. فبدلاً من إنشاء 30 مهمة منفصلة لتشغيل الأمر نفسه خلال أول 30 دقيقة من كل ساعة مثل ( ..... * * * * 1 , ..... * * * * 2 , ..... * * * * 0 إلخ ) يمكن كتابة المدى مباشرة ( ..... * * * * 0-29 ) |
| /     | يمكن استخدامها لتحديد خطوة زمنية. على سبيل المثال، بدلاً من كتابة 8 مهام منفصلة لتشغيل أمر ما كل 3 ساعات مثل ( ..... * * * * 0 3 , ..... * * * * 0 6 , ..... * * * * 0 0 ) يمكن كتابة ( ..... * * * * 0 / 3 )                                  |

| المعنى                                                                                            | تعبير Cron         |
|---------------------------------------------------------------------------------------------------|--------------------|
| ينفذ الأمر مرة كل دقيقة، طوال اليوم، كل يوم.                                                      | * * * * *          |
| ينفذ الأمر عند الدقيقة 12 من كل ساعة، مثل 1:12، 2:12، 3:12، إلخ.                                  | 12 * * * *         |
| ينفذ الأمر عند الدقائق 00، 15، 30، و45 من كل ساعة.                                                | 0,15,30,45 * * * * |
| نفس المثال السابق، شكل مختصر يعني "كل 15 دقيقة".                                                  | */15 * * * *       |
| ينفذ الأمر مرة واحدة يوميًا عند الساعة 4 صباحًا.                                                  | 0 4 * * *          |
| ينفذ الأمر الساعة 4 صباحًا فقط في أيام الثلاثاء (2)، الأربعاء (3)، والخميس (4).                   | 0 4 * * 2-4        |
| ينفذ الأمر عند الدقائق 20 و40 من كل ساعة ثامنة (00:00، 08:00، 16:00) خلال الأشهر 7-12 (JUL - DEC) | 20,40 */8 * 7-12 * |

### مثال توضيحي:

نريد إنشاء مهمة Cron تقوم بتنفيذ الأمر echo، ثم حفظ ناتج هذا الأمر داخل الملف: /cronTest/hello.txt. قمنا بإنشاء مجلد باسم cronTest، وهو في هذه المرحلة مجلد فارغ سيتم استخدامه لتخزين مخرجات المهمة:

```
rbcs@tasneem-VMware-Virtual-Platform:~/cronTest$ ls
rbcs@tasneem-VMware-Virtual-Platform:~/cronTest$
```

بعد إنشاء المجلد، قمنا بإضافة المهمة إلى ملف crontab باستخدام الأمر: crontab -e يقوم هذا الأمر بفتح محرر النصوص الافتراضي، أضفنا في نهاية الملف السطر الذي يمثل مهمة Cron، وتم ضبط المهمة بحيث تنفذ كل دقيقة، وذلك لسهولة متابعة ظهور النتيجة بشكل سريع.

بعد الانتهاء من كتابة المهمة، قمنا بحفظ ملف crontab باستخدام أوامر التحرير في محرر vim عن طريق الضغط على esc ثم كتابة wq: ثم الضغط على enter ليتم حفظ المهمة و تفعيلها

```
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
* * * * * echo 'hello world' >> cronTestt/hello.txt
-- INSERT --
```

25,52

Bot

بعد مرور دقيقة من إضافة المهمة، نلاحظ أن الملف hello.txt قد تم إنشاؤه داخل مجلد cronTest، وأنه يحتوي على ناتج الأمر.

```
rbcs@tasneem-VMware-Virtual-Platform:~/cronTest$ ls
hello.txt
rbcs@tasneem-VMware-Virtual-Platform:~/cronTest$ cat hello.txt
hello world
rbcs@tasneem-VMware-Virtual-Platform:~/cronTest$
```

هذا يؤكد أن مهمة Cron تم تنفيذها بنجاح وفق الجدول الزمني المحدد (كل دقيقة).

#### ■ أمثلة عملية على استخدام تعبيرات Cron

| تعبير cron                                                                  | التوقيت                           | الأمر المُنفذ                             | شرح الأمر                                                                                                                                                           |
|-----------------------------------------------------------------------------|-----------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 * * * * df-h ><br>~/logs/disk_usage_\$(date<br>+\\%Y\\%m\\%d\\%H\\%M).log | كل ساعة، عند<br>الدقيقة 0         | df-h                                      | تسجيل استخدام القرص في ملف يحتوي<br>على طابع زمني داخل مجلد ~/logs (مثال:<br>disk_usage_20251106_0100.log<br>يتم استخدام المحرف % مع الهروب لأنه<br>ضروري داخل cron |
| * / 10 * * * * ping -c 4 8.8.8.8 >><br>~/logs/ping_log.txt                  | كل 10 دقائق                       | ping -c 4<br>8.8.8.8                      | إرسال 4 طلبات ping إلى خادم Google<br>DNS (8.8.8.8) و تسجيل النتائج في ملف<br>ping_log.txt                                                                          |
| 0 3 * * * sudo systemctl restart<br>apache2                                 | يوميًا الساعة<br>3:00 صباحًا      | systemctl<br>restart<br>apache2           | إعادة تشغيل خادم الويب Apache2 تلقائيًا<br>يوميًا                                                                                                                   |
| 0 0 * * 0 sudo find /tmp-type f<br>mtime +7 -delete                         | كل يوم أحد<br>عند 12:00<br>صباحًا | find /tmp-<br>type f -mtime<br>+7 -delete | حذف الملفات الموجودة في /tmp والتي مرّ<br>عليها أكثر من 7 أيام بهدف توفير مساحة.                                                                                    |
| 0 * * * * free -h >><br>~/logs/memory_usage.log                             | كل ساعة عند<br>الدقيقة 0          | free -h                                   | تسجيل استخدام الذاكرة بصيغة سهلة<br>القراءة في ملف memory_usage.log                                                                                                 |
| * * * * * date >><br>~/logs/time_log.txt                                    | كل دقيقة                          | date                                      | تضيف التاريخ والوقت الحاليين لمحتوى<br>الملف time_log.txt                                                                                                           |
| * / 30 * * * * md5sum<br>/etc/passwd >><br>~/logs/passwd_checksum.log       | كل 30 دقيقة                       | md5sum<br>/etc/passwd                     | حساب بصمة (Checksum) ملف<br>/etc/passwd وتسجيلها للتحقق من أي<br>تغييرات قد تشير إلى اختراق أو تعديل غير<br>مقصود                                                   |

## شرح تعليمية md5sum

تستخدم أداة md5sum لحساب قيمة تجزئية (Checksum) لملف/مجلد ما، بهدف التحقق من سلامته والتأكد من عدم حدوث أي تعديل عليه، وتعتمد هذه الأداة على خوارزمية MD5 التي تقوم بإنتاج رقم مميز (بصمة رقمية) يمثل محتوى الملف/المجلد.

و يجب الانتباه أن خوارزمية MD5 لا توفر سرية أو حماية أمنية عالية، ولا تُستخدم للتشفير، بل تُستخدم فقط للكشف عن التغييرات، حيث أن أي تعديل - حتى لو كان بسيط جداً - على الملف أو المجلد يؤدي إلى تغيير البصمة الناتجة بشكل كامل.

## مثال تطبيقي لتعليمية md5sum:

قمنا بإضافة مهمة في ملف crontab لتشغيل الأمر التالي كل دقيقة:

```
md5sum /etc/passwd >> ~/logs/passwd_checksum.log
```

حيث يتم حساب البصمة الرقمية لملف /etc/passwd وتسجيلها في ملف السجل داخل المسار  
~/logs/passwd\_checksum.log

```
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
* * * * * md5sum /etc/passwd >> ~/logs/passwd_checksum.log
-- INSERT --
```

25,59

Bot

- الرمز " ~ " يشير إلى المجلد الأساسي للمستخدم (Home Directory)، فعند تنفيذ الأمر: `cd ~` ينتقل النظام إلى المجلد الرئيسي للمستخدم داخل مجلد `/home`.
- يجب أن يكون المجلد `logs` موجوداً وإلا لن ينفذ الأمر، لأن المجلدات لا يتم إنشاؤها بشكل تلقائي إذا لم تكن موجودة مسبقاً.
- أما بالنسبة للملف `passwd_checksum.log` فليس بالضرورة أن يكون موجوداً مسبقاً حيث يتم إنشاء الملفات تلقائياً إذا لم تكن موجودة.



## نملك الخيار وخيارنا الأمل ...

بعد مرور دقيقة، نلاحظ أن الملف passwd\_checksum.log أصبح يحتوي على قيمة تجزئية (Checksum) للملف /etc/passwd.

```
rbc@tasneem-VMware-Virtual-Platform:~/logs$ cat passwd_checksum.log
1b83a64bebfc1350e00f2cf4f125b89b /etc/passwd
```

سنقوم بالتعديل على ملف ال /etc/passwd وذلك عن طريق إضافة مستخدم جديد (ويمكن حذف أو تعديل معلومات مستخدم قديم أيضاً، يعتبر تعديل على المجلد)، فنلاحظ أن البصمة الرقمية الناتجة من md5sum تغيرت، مما يؤكد حدوث تعديل في الملف.

```
rbc@tasneem-VMware-Virtual-Platform:~/logs$ cat passwd_checksum.log
1b83a64bebfc1350e00f2cf4f125b89b /etc/passwd
b7589b6736cb694abca36cd49e435fe0 /etc/passwd
```

## Ubuntu Network Configuration

### ■ إدارة الشبكة في أوبونتو :

أوبونتو يستخدم Netplan كمدير الشبكة الافتراضي، وملفات الإعدادات مخزنة في المسار /etc/netplan/ ونعرض محتوياته باستخدام الأمر ls /etc/netplan هذا الأمر سيجع أسماء ملفات التهيئة ذات الامتداد .yaml. يمكن تعديل إعدادات ال netplan باستخدام محرر النصوص، مثلاً:

- تعديل الملف باستخدام nano : `sudo nano /etc/netplan/filename.yaml`
- تعديل الملف باستخدام vim : `sudo vim /etc/netplan/filename.yaml`

### ■ عرض عنوان IP الحالي :

يمكن استخدام أحد الأمرين التاليين لعرض عناوين IP الخاصة بالجهاز ومعلومات أخرى تفصيلية :

`ip a , ip addr`



## مثال توضيحي شامل لإضافة Static IP باستخدام Netplan (Ubuntu Desktop)

### 1. التحقق من عنوان الـ IP الحالي:

بدايةً نقوم بعرض عنوان الـ IP الموجود حالياً باستخدام التعليمة: `ip a`

```
rbc@ -VMware-Virtual-Platform:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:a3:22:73 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.1.133/24 brd 192.168.1.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
    inet 192.168.87.128/24 brd 192.168.87.255 scope global dynamic noprefixroute ens33
        valid_lft 1009sec preferred_lft 1009sec
```

### 2. فتح ملف إعدادات Netplan

بما أن نسخة Ubuntu Desktop لا تحتوي على إعدادات static IP في ملف netplan بشكل افتراضي، سنقوم بإضافة الإعدادات يدوياً.

نفتح الملف باستخدام محرر النصوص مثلاً `vim` :

```
sudo vim /etc/netplan/01-network-manager-all.yaml
```

### 3. إضافة إعدادات Static IP

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.1.13/24
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
```

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.1.13/24
      routes:
        - to: 0.0.0.0/0
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
```



Linux hides nothing  
everything is there if you look for it

إذا استعملنا الصيغة في الصورة اليسارية قد يعطي التنبيه الآتي:

```
** (process:3723): WARNING **: 15:44:40.252: `gateway4` has been deprecated, use
default routes instead.
See the 'Default routes' section of the documentation for more details.
```

وهذا لأنها صيغة قديمة تعتمد تنسيقاً غير مدعوم في الإصدارات الحديثة، ونقوم بحل هذه المشكلة باتباع الصيغة في الصورة على اليمين.

#### 4. تجربة الإعدادات قبل تطبيقها نهائياً

بعد حفظ الملف، نريد التأكد من أن الإعدادات سليمة. لذلك نستخدم الأمر: `sudo netplan try` يقوم هذا الأمر بتجربة الإعدادات مؤقتاً، وسيطلب الضغط على Enter لتأكيد التغييرات خلال مدة زمنية محدودة. وإذا لم يتم التأكيد، سيعود النظام تلقائياً للإعدادات السابقة للحماية من فقدان الاتصال.

```
rbcs@ -VMware-Virtual-Platform:~$ sudo netplan try
Do you want to keep these settings?

Press ENTER before the timeout to accept the new configuration

Changes will revert in 120 seconds
Configuration accepted.
```

بعد تأكيد الإعدادات، نعيد تنفيذ `ip a` وسنلاحظ تغيير عنوان الـ IP إلى العنوان الذي قمنا بتحديدده في إعدادات `netplan`

```
rbcs@ -VMware-Virtual-Platform:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:a3:22:73 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.1.13/24 brd 192.168.1.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
    inet 192.168.87.128/24 brd 192.168.87.255 scope global dynamic noprefixroute ens33
        valid_lft 1797sec preferred_lft 1797sec
```

#### إضافة (DHCP) Dynamic IP بدل Static IP

في المثال السابق قمنا بإضافة static IP لذلك وضعنا `dhcp4: no`، إذا أردنا إعطاء dynamic IP نضع:

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    eth0:
      dhcp4: yes
~
```



#### ■ الأمر : `sudo netplan try`

يُستخدم لتجربة إعدادات الشبكة بشكل مؤقت قبل تطبيقها بشكل نهائي.  
بعد تنفيذ الأمر، يطلب النظام منك تأكيد التغييرات خلال مدة زمنية محددة.  
إذا لم يتم تأكيد التعديلات، يقوم النظام تلقائياً بالعودة إلى الإعدادات السابقة حفاظاً على الاتصال.

#### ■ الأمر : `sudo netplan apply`

يقوم بتطبيق إعدادات الشبكة مباشرةً ودون تجربة مؤقتة، أي يتم اعتمادها فوراً.  
عند استخدام الأمر `netplan try` والموافقة على التعديلات، لا حاجة لاستخدام الأمر `netplan apply` بعدها، لأن التغييرات ستطبق تلقائياً بعد التأكيد.

#### تذكر :

ما هو ال DHCP ؟

هو الخدمة التي تعطي الجهاز IP تلقائياً بدون كتابته يدوياً، أي أنه نظام توزيع عناوين IP تلقائياً داخل الشبكة.

ما هو ال Gateway ؟

هو عنوان الجهاز (البوابة / المنفذ) الذي يربط الشبكة الداخلية بالإنترنت، وغالباً يكون هذا الجهاز هو الراوتر.

#### فائدة أرييسيزية :

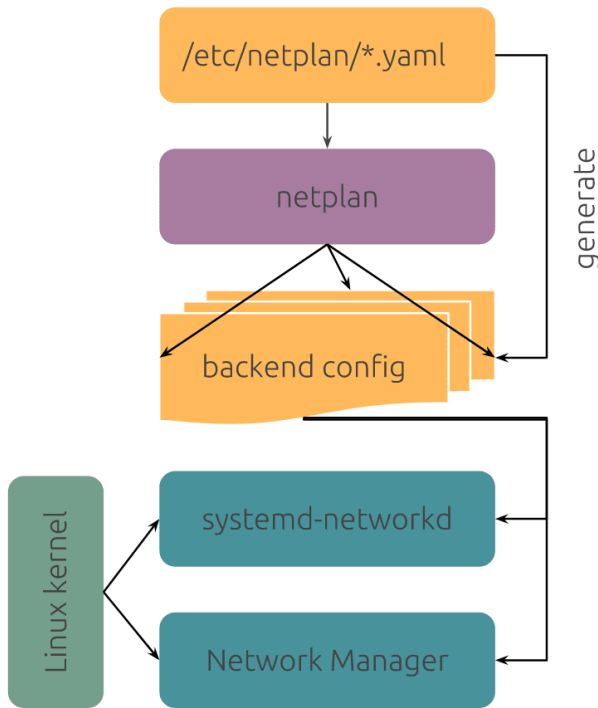
#### ■ ما هو Netplan ؟

Netplan هو نظام لإدارة وإعداد الشبكات (Network Configuration) في إصدارات أوبونتو الحديثة.  
وهو يعتمد على ملفات إعداد مكتوبة بصيغة YAML الموجودة في المسار: `/etc/netplan/`

#### ■ فكرة عمل Netplan:

1. يقوم المستخدم بكتابة إعدادات الشبكة داخل ملف YAML  
(نوع الاتصال, Static IP, DHCP, أسماء الشبكات اللاسلكية ....).
2. يقوم Netplan بترجمة هذه الإعدادات إلى تنسيق تستخدمه خدمة أخرى في الخلفية (backend) مثل:  
a) NetworkManager (غالباً في نسخة Desktop)  
b) systemd-networkd (غالباً في نسخة Server)

بمعنى آخر: Netplan هو الطبقة التي نكتب فيها الإعدادات، بينما أدوات الشبكة الأساسية هي التي تنفذ الإعداد فعلياً.



3. systemd-networkd / NetworkManager  
بعد أن ينشئ Netplan الإعدادات، يقوم أحد هذين البرنامجين بتطبيق إعدادات الشبكة على:

- العناوين
- DHCP
- routes
- DNS

4. Kernel  
في النهاية، تعمل الشبكة فعليًا على مستوى نواة لينكس حيث يتم تطبيق الإعدادات فعليًا على الطبقة الفيزيائية.

#### ■ Ubuntu Desktop Network :

يستخدم NetworkManager كمدير الشبكة الرئيسي، وهو مناسب لأجهزة المستخدمين لأنه:

- يدعم الاتصال اللاسلكي Wi-Fi بسهولة.
- يوفر واجهة رسومية (GUI) لتغيير الإعدادات بسرعة.
- يسمح بوجود عدة شبكات محفوظة وتبديلها تلقائيًا

```
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
~
~
```

لا يوجد إعدادات بطاقة الشبكة (Ethernet/Wi-Fi) داخل Netplan لأن NetworkManager هو الذي يدير كل شيء.

#### ■ Ubuntu Server Network :

يستخدم systemd-networkd لإدارة الشبكة، وهو مناسب للسيرفرات لأنه:

- أكثر استقرارًا.
- مثالي للإعدادات الثابتة (static) مثل IP ثابت
- ونجد تعريف بطاقة الشبكة داخل ملف Netplan، مثل:

```
GNU nano 4.8 /etc/netplan/00-installer-config.yaml
# This is the network config written by 'subiquity'
network:
  ethernets:
    ens33:
      dhcp4: true
  version: 2
```